

Verilog 語法入門 — 給寫 Python 的人

在讀第 1 週之前先看這份。全部用 Python 對照，你熟悉的東西在 Verilog 裡叫什麼、哪些符號意思完全不一樣。

目錄

- 0. 三個一定要先知道的差異
 - 1. 符號對照表
 - 2. 註解與分號
 - 3. 數字怎麼寫
 - 4. 大括號 `{}` — 跟 Python 完全不同
 - 5. 變數：`wire` 和 `reg`
 - 6. `for` 迴圈 — 它不是迴圈
 - 7. `if` / `case`
 - 8. `module` — 相當於什麼
 - 9. `always` 區塊
 - 10. `=` 和 `<=`
 - 11. `function` / `task`
 - 12. 印字：`$display`
 - 13. Python 沒有的東西
 - 14. 動手跑一次
-

0. 三個一定要先知道的差異

差異 1：所有東西同時發生

```
# Python：由上而下，一次做一件事
a = 1
b = a + 1
c = b + 1
```

```
// Verilog：這三行「同時」存在，沒有先後
assign a = 1;
assign b = a + 1;
assign c = b + 1;
```

把三行上下對調，Verilog 的結果完全一樣。因為它們描述的是三段實體電路，電流不會排隊。

這是最大的思維轉換。你不是在寫「步驟」，是在描述「有哪些東西存在」。

差異 2：所有東西都有固定的位元寬度

```
x = 200          # Python 自動處理，想多大就多大
x = x * 10**50   # 也沒問題
```

```
wire [7:0] x;    // ★ 8 條實體的線，最多裝 0~255
assign x = 300;  // 裝不下 → 被截斷成 44，而且不會警告你
```

寫 Verilog 就是在決定「要拉幾條線」。每個訊號都要先講清楚幾位元。

差異 3：縮排完全沒有意義

```
if x:
    do_this()      # 縮排決定了它屬於 if
do_that()          # 這行不屬於 if
```

```
if (x)
    do_this;       // 縮排只是給人看的
do_that;           // ★ 這行本來就不屬於 if（因為 if 只管一句）
```

Verilog 只認分號和 **begin / end** 。要包多句就要寫 **begin / end** :

```
if (x) begin
    do_this;
    do_that;    // 這樣兩句都屬於 if
end
```

1. 符號對照表

你想做的事	Python	Verilog
註解	#	// (# 在 Verilog 是延遲！)
多行註解	"""..."""	/* ... */
程式區塊	縮排	begin / end
一句結束	換行	; 分號
等於 (比較)	==	== 或 ===
賦值	=	= 或 <= (差很多，見第 10 節)
且 / 或 / 非	and or not	&& !
位元且/或/互斥 或	& ^	& ^ (一樣)
位元反	~	~
一整條線縮成一個值	all() / any()	&a / a (縮減，見第 13 節)
位移	<< >>	<< >> 外加 <<< >>> (有號數專用，見第 13 節)
三元運算	a if c else b	c ? a : b
印東西	print()	\$display()
函式	def	function / task
類別 / 模組	class	module
串接	+ (字串) / list1+list2	{a, b}
重複	"ab" * 3	{3{a}}
取索引	a[0]	a[0] (一樣)
取切片	a[0:4]	a[3:0] (高位在前！)

⚠️ 最容易踩的兩個：

1. # 在 Verilog 是時間延遲，不是註解
2. 切片是 [高:低]，跟 Python 的 [起:迄] 順序相反

2. 註解與分號

```
// 單行註解

/* 多行
   註解 */

assign y = a & b;      // ← 每一句都要分號
endmodule              // ← 但 endmodule / endfunction / end 後面不用
```

初學九成的語法錯誤都是漏分號。而且錯誤訊息會指到下一行：

```
01_gates.v:12: syntax error
```

第 12 行看起來沒問題？去看第 11 行是不是忘了分號。

3. 數字怎麼寫

Python 裡 10 就是 10。Verilog 裡你還要講「用幾條線裝」。

```
8'b1010_0101    // 二進位
8'hA5           // 十六進位
8'd165          // 十進位
8'o245          // 八進位
```

四個都是同一個值 165。格式固定：

8	'	h	A5
↑	↑	↑	↑
幾位元	分隔符	進位	值

進位字母	意思
b	binary 二進位
o	octal 八進位
d	decimal 十進位
h	hex 十六進位

底線 `_` 只是給人看的，跟 Python 的 `1_000_000` 一樣，電腦會忽略。

為什麼要寫寬度？

```
wire [3:0] x;
assign x = 8'd200;           // 200 需要 8 位元，但 x 只有 4 位元
                             // → 只留低 4 位 = 8，而且不警告你
```

這是 Python 人最常踩的坑。Python 的整數無限大，Verilog 的每個值都是固定條數的線。

可以偷懶嗎？

可以，但別養成習慣：

```
assign x = 5;                // 沒寫寬度，會自動當成 32 位元再截斷
assign x = 4'd5;             // ✅ 建議這樣寫，意圖明確
```

4. 大括號 `{}` —— 跟 Python 完全不同

```
d = {"a": 1}                 # Python: dict
s = {1, 2, 3}                 # Python: set
```

```
{a, b}                       // Verilog: 把 a 和 b 這兩束線「黏成一束」
```

Verilog 的 `{}` 是「接線」，不是資料結構，也不是程式區塊。

四種用法

先把兩個輸入值寫清楚（這是 `02_syntax_tb.v` 裡實際用的值）：

```
a = 8'b1100_0001    // 8 位元
b = 8'b0000_1111    // 8 位元
```

寫法	結果	幾位元
<code>{a, b}</code>	<code>1100000100001111</code>	16
<code>{2{a}}</code>	<code>1100000111000001</code>	16
<code>{a[0],a[1],...,a[7]}</code>	<code>10000011</code>	8
<code>{a[7:4], 4'b1010}</code>	<code>11001010</code>	8

下面一個一個推給你看。

① `{a, b}` —— 接起來

```
a =      1100 0001
b =              0000 1111
      ↓          ↓
{a, b} = 1100 0001 0000 1111    ← 16 位元
          └─ a ─┘ └─ b ─┘
          (高位)  (低位)
```

左邊寫的在高位，跟你唸數字的方向一樣。 $8 + 8 = 16$ 位元。

⚠ 這不是加法。`{a, b}` 不等於 `a + b`，它是「把兩束線並排接成一束」。

② `{2{a}}` —— 複製

```
{2{a}} = {a, a}
        = 1100 0001 1100 0001    ← 16 位元
          └─ a ─┘ └─ a ─┘
```

`{2{a}}` 裡面的 2 是「重複幾次」。`{3{a}}` 就是 24 位元。

注意有兩層大括號：外層是「串接」，內層 `{a}` 是「要重複的東西」。寫成 `{2 a}` 是語法錯誤。

③ `{a[0],a[1],...,a[7]}` —— 前後顛倒

先把 `a` 的每一位標上編號（右邊是第 0 位）：

```
a = 1 1 0 0 0 0 0 1
位置 7 6 5 4 3 2 1 0
```

顛倒 = 把第 0 位放到最左邊，第 7 位放到最右邊：

```
{ a[0], a[1], a[2], a[3], a[4], a[5], a[6], a[7] }
  ↓      ↓      ↓      ↓      ↓      ↓      ↓      ↓
  1      0      0      0      0      0      1      1
= 1000 0011                                ← 8 位元
```

對照：原本 `11000001` → 顛倒後 `10000011`。眼睛看得出來是左右鏡射。

④ `{a[7:4], 4'b1010}` —— 夾常數

```
a[7:4] = 1100      ← 取 a 的高 4 位
4'b1010 =      1010 ← 直接寫死的常數
```

```
結果    = 1100 1010                                ← 8 位元
          └高4位┐ └常數┘
```

串接裡面可以放任何東西：整條線、切片、常數，混著用都行，只要總位元數對得起來就好。

用 Python 類比

```
a_bits = "11000001"
b_bits = "00001111"

# {a, b}    -> 接起來
a_bits + b_bits          # "1100000100001111"

# {2{a}}    -> 複製
a_bits * 2               # "1100000111000001"

# 顛倒
a_bits[::-1]             # "10000011"

# 夾常數
a_bits[0:4] + "1010"     # "11001010"
```

但注意：Python 是在處理「字串」，Verilog 是在「實體接線」。Verilog 的 `{}` 不產生任何邏輯閘——它只是決定哪條線接到哪裡，**完全免費**。

位元寬度一定要對得起來

```
wire [15:0] y1;
assign y1 = {a, b};           // ✅ 8+8 = 16，剛好

wire [7:0] y2;
assign y2 = {a, b};           // ⚠️ 16 位元塞進 8 位元 → 只留低 8 位 = b

wire [15:0] y3;
assign y3 = {3{a}};           // ⚠️ 24 位元塞進 16 位元 → 高位被砍掉
```

Verilog 不會報錯，會默默截斷。這是 Python 人最容易踩的坑（Python 的 list 想多長就多長）。

這也是 SYNTAX.md 第 14 節「改改看」第 2 題要你試的東西——把 `{2{a}}` 改成 `{3{a}}`，親眼看它被砍掉。

那程式區塊用什麼？

begin / end :

```
if (x) begin          // ← 不是 {
    y = 1;
    z = 2;
end                  // ← 不是 }
```

5. 變數：**wire** 和 **reg**

Python 的變數沒有型別。Verilog 有兩種，而且選錯會編譯錯誤。

```
wire [7:0] a;        // 一條線，值由「別人」持續驅動
reg  [7:0] b;        // 一個可以「被寫入」的東西
```

怎麼選？看你怎麼給它值：

你用什麼給值	就宣告成
<code>assign y = ...</code>	wire
在 <code>always</code> 或 <code>initial</code> 裡面 <code>y = ...</code>	reg

```
wire y1;
assign y1 = a & b;           // assign → wire ✓

reg y2;
always @(*) begin
    y2 = a & b;              // always 裡 → reg ✓
end
```

[!WARNING] **reg** 這個名字取得很爛。它不代表會變成正反器（暫存器）。

`always @(*)` 裡的 **reg** 綜合出來是純組合邏輯，一顆正反器都沒有。就把它當成「一個可以被賦值的變數」。

位元寬度

```
wire      x;          // 1 條線
wire [7:0] y;          // 8 條線，編號 7 6 5 4 3 2 1 0
```

`[7:0]` 讀作「第 7 位到第 0 位」。高位寫前面，跟 Python 切片相反。

```
y[7]      // 最高位 (MSB)
y[0]      // 最低位 (LSB)
y[7:4]    // 高 4 位
y[3:0]    // 低 4 位
```

6. for 迴圈 —— 它不是迴圈

這是 Python 人最需要重新理解的一個。

```
# Python：跑 8 次，花 8 個時間單位
total = 0
for i in range(8):
    total += bits[i]
```

```
// Verilog：綜合成 7 個加法器「串成一條鏈」，一個時脈之內走完
count = 0;
for (i = 0; i < 8; i = i + 1)
    count = count + x[i];
```

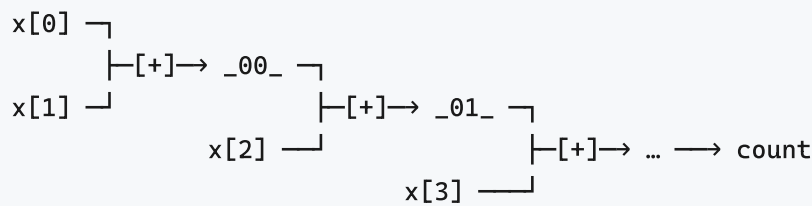
它到底變成什麼？

Verilog 會把迴圈完全展開。用 Yosys 實際綜合這段，產生的是：

```
assign _00_ = {3'h0, x[0]} + x[1];    // 第 1 個加法器
assign _01_ = _00_ + x[2];           // 吃前一個的輸出
assign _02_ = _01_ + x[3];
assign _03_ = _02_ + x[4];
assign _04_ = _03_ + x[5];
assign _05_ = _04_ + x[6];
assign count = _05_ + x[7];          // 第 7 個加法器
```

7 個加法器（8 個數相加只要 7 次加法），而且每一個的輸入都是前一個的輸出。

實體上長這樣：



訊號從左邊進去，像水流一樣穿過 7 個加法器，從右邊出來

[!!IMPORTANT] 這裡有兩件事要分清楚：

說法	對不對
7 個加法器同時存在	✓ 電路裡真的有 7 份實體加法器
7 次加法同時發生	✗ 有相依性，必須一個接一個
一個時脈之內算完	✓ 訊號穿過整條鏈，中間沒有正反器

「不用多個時脈」不等於「沒有先後」。電子從左流到右要花時間（傳播延遲），但那是物理現象，跟時脈無關。

代價：這條鏈有多長？

7 個加法器串起來 → 延遲是單個加法器的 7 倍
延遲越長 → 時脈就要拉越慢

假設一個加法器 1ns，這條鏈 7ns → 這個電路最快只能跑 143 MHz。

接成樹狀會快很多：

串鏈（7 層深）：

$x_0 + x_1$
+ x_2
+ x_3
+ x_4
+ x_5
+ x_6
+ x_7

延遲 = 7 個加法器

樹狀（3 層深）：

$x_0 + x_1$ $x_2 + x_3$ $x_4 + x_5$ $x_6 + x_7$ ← 4 個真的同時
└───┘ └───┘
+ + ← 2 個同時
└───┘
+ ← 1 個

延遲 = 3 個加法器

同樣 7 個加法器，接法不同，速度差兩倍多。這條「最長的路」叫**關鍵路徑**，第 7 週和第 12 週會專門處理它。

語法上的差別：

	Python	Verilog
寫法	<code>for i in range(8):</code>	<code>for (i = 0; i < 8; i = i + 1)</code>
迴圈變數	自動	要先宣告 <code>integer i;</code>
有 <code>range()</code> 嗎	有	沒有，三段式自己寫
包多句	縮排	<code>begin / end</code>

觀念上的差別（更重要）：

Python 的 `for` → 「重複做 N 次」，花 N 倍時間
Verilog 的 `for` → 「複製 N 份硬體」，全部同時算完

所以 Verilog 的 `for` 次數必須在編譯時就能算出來（不能是變數），因為它要決定「複製幾份」。

```
for (i = 0; i < 8; i = i + 1)           //  8 是常數  
for (i = 0; i < n; i = i + 1)         //  n 是訊號，綜合工具不知道要複製幾份
```

代價：複製 8 份就佔 8 份面積。Python 的迴圈只佔一份程式碼、跑 8 次；
Verilog 是用面積換速度。

7. if / case

if

```
if a == 1:
    y = 2
elif a == 2:
    y = 3
else:
    y = 0
```

```
if (a == 1)           // ← 條件要括號
    y = 2;
else if (a == 2)      // ← 是 else if，不是 elif
    y = 3;
else
    y = 0;
```

多句要 `begin / end` :

```
if (a == 1) begin
    y = 2;
    z = 5;
end
```

case (Python 3.10 的 `match`)

```
case (sel)
    2'd0:    y = a;
    2'd1:    y = b;
    2'd2:    y = c;
    default: y = 0;    // ★ 一定要寫！
endcase
```

[!IMPORTANT] **default** 一定要寫，**if** 也一定要有 **else**。組合邏輯裡漏掉任何一條路徑，Verilog 會幫你生出一個「記住上次的值」的元件 (latch)，那是設計大忌。第 3 週會專門講。

8. module — 相當於什麼

最接近的是 `class`，但更像是「一顆 IC 的接腳定義」。

```
class Counter:
    def __init__(self, clk, reset):
        self.clk = clk
        ...
```

```
module counter (
    input wire clk,           // 進來的腳
    input wire reset,
    output reg [3:0] count    // 出去的腳
);
    // 內容
endmodule
```

用它 = 實例化（相當於 Python 的建立物件）：

```
c1 = Counter(clk, reset)    # Python
```

```
counter uut (                // uut 是這一顆的名字
    .clk(clk),                // .那顆的腳位(我這邊的線)
    .reset(reset),
    .count(count)
);
```

`.clk(clk)` 唸成「把 counter 的 `clk` 腳，接到我的 `clk` 線上」。就像插排線——左邊是 IC 的腳，右邊是你的杜邦線。

可以建立很多顆：`counter u1 (...); counter u2 (...);` 這時候真的會有兩份實體電路，不是共用一份程式碼。

9. always 區塊

Python 沒有對應的東西。它是「什麼時候要重算」的宣告。

```
// 組合邏輯：只要右邊任何東西變了，就重算
always @(*) begin
    y = a & b;
end

// 循序邏輯：只在 clk 從 0 變 1 的那一瞬間動作
always @(posedge clk) begin
    q <= d;
end
```

寫法	什麼時候執行	綜合成
<code>always @(*)</code>	輸入一變就重算	邏輯閘（沒有記憶）
<code>always @(posedge clk)</code>	只在時脈上升緣	正反器（有記憶）
<code>initial</code>	模擬開始時執行一次	不能合成，只能測試用

還有 `assign`：

```
assign y = a & b;      // 跟 always @(*) 效果一樣，但只能寫一句
```

一句就用 `assign`，多句才用 `always @(*)`。

10. = 和 <=

Python 只有一個 `=`。Verilog 有兩個，而且做出來的電路不一樣。

```
q1 <= d;      // 非阻塞：先把右邊全部讀好，再一起寫左邊
q2 = d;       // 阻塞：這句做完才做下一句（像 Python）
```

規則：

寫在哪	用哪個	為什麼
<code>assign y = ...</code>	只能 <code>=</code>	那是「連續指定」，不是阻塞賦值，也不能寫 <code><=</code>
<code>always @(*)</code>	<code>=</code>	後面那行要看到前面剛算好的中間值
<code>initial</code>	<code>=</code>	像寫腳本，一步做完再做下一步
<code>always @(posedge clk)</code>	<code><=</code>	所有正反器要同時抓「上一拍」的值

是的——`always @(*)` 和 `initial` 裡的賦值就是阻塞的，行為跟 Python 一樣。

為什麼各自要用那一種

用實體來想：

組合邏輯：訊號一路穿過閘 → 前面算完後面才有得算 → 阻塞 `=`
時脈邏輯：所有正反器同時被敲一下 → 大家都抓「舊值」 → 非阻塞 `<=`

兩邊用錯的後果剛好相反：

用錯的地方	後果	哪個範例
時脈區塊用了 <code>=</code>	3 顆正反器塌成 1 顆	第 3 週 04_blocking_vs_non
組合區塊用了 <code><=</code>	輸出先給錯值再追上 → 毛刺	第 3 週 06_comb_blocking

06 的實測輸出（電路是 `y = (a & b) | c`）：

```

a=1 b=1 c=0 | 剛設定完：y_ok=1  y_bad=0    ← y_bad 錯的
              | 等 5ns 後：y_ok=1  y_bad=1    ← 才追上

a=0 b=0 c=0 | 剛設定完：y_ok=0  y_bad=1    ← 又錯
              | 等 5ns 後：y_ok=0  y_bad=0    ← 才追上

```

`y_bad` 每次都先閃一個錯的值。原因是 `<=` 會「先把右邊全部讀好」，所以第二句讀中間值的時候，第一句還沒寫進去——讀到的是上一次的。

`<=` 在時脈區塊裡不是「小於等於」，是賦值。比較大小要用 `<=` 的時候，只會出現在 `if (a <= b)` 這種括號裡的條件式，不會跟賦值搞混。

11. function / task

```
def add(a, b):  
    return a + b
```

```
function [7:0] add;           // [7:0] 是回傳值寬度  
    input [7:0] a, b;        // 參數  
    begin  
        add = a + b;         // ★ 用「函式名 = 值」回傳，沒有 return  
    end  
endfunction
```

呼叫方式一樣：`y = add(x, z);`

	function	task
回傳值	一定要有一個	沒有
可以有延遲 #	✗	✓
能不能變成硬體	✓	通常不行
用在哪	設計裡的運算	測試平台的重複步驟

12. 印字：`$display`

```
print(f"a={a} b={b}")
```

```
$display("a=%b b=%b", a, b);
```

`%b` 是留一個空位，後面的變數依序填進去（跟 C 的 `printf` 一樣）。

格式	印成
<code>%b</code>	二進位 <code>1010</code>
<code>%d</code>	十進位 <code>10</code>
<code>%h</code>	十六進位 <code>a</code>
<code>%0d</code>	十進位但不補空格
<code>%0t</code>	目前的模擬時間

`$display` 只能用在測試平台 (`initial` / `always` 裡)，不能合成。所有 `$` 開頭的都是「給模擬器的指令」，硬體看不到。

13. Python 沒有的東西

這幾個沒有對照，要重新學：

東西	意思
<code>#10</code>	等 10 個時間單位 (只能用在測試檔)
<code>posedge</code> / <code>negedge</code>	訊號的上升緣 / 下降緣
<code>wire</code> vs <code>reg</code>	見第 5 節
<code>x</code>	「不知道」—— 變數沒給初值時的狀態
<code>z</code>	「浮空」—— 這條線沒接東西
<code>`timescale</code>	定義 <code>#1</code> 是多久
位元寬度	每個訊號都要決定用幾條線
縮減運算子 <code>&a</code> <code> a</code> <code>^a</code>	把一整條線縮成 1 個位元 (見下面)
算術右移 <code>>>></code>	有號數專用的右移，左邊補符號位 (見下面)

`x` 和 `z` 特別重要 —— 它們是你以後除錯最主要的線索：

- 波形一片紅色 `x` → 忘了給初值 / reset 沒接好
- 波形出現中線 `z` → 忘了接線

★ 縮減運算子 —— Python 要寫迴圈，Verilog 一個符號

這個 Python 完全沒有對應寫法，第一次看到一定會愣住。

符號放在變數「前面」，只有一個運算元，意思是「把這個符號插到這條線的每個位元中間，全部算成一個位元」：

```
wire [3:0] a = 4'b1100; // a[3]=1 a[2]=1 a[1]=0 a[0]=0

&a    →  1 & 1 & 0 & 0  =  0
|a    →  1 | 1 | 0 | 0  =  1
^a    →  1 ^ 1 ^ 0 ^ 0  =  0
```

用 Python 想的話：

```
a = [1, 1, 0, 0]

&a    相當於    all(a)           # 全部都是 1 嗎？        -> False (0)
|a    相當於    any(a)           # 有沒有任何一個 1？    -> True  (1)
^a    相當於    sum(a) % 2 == 1  # 1 的個數是奇數嗎？   -> False (0)
```

Python 要跑迴圈，Verilog 一個符號就是一棵閘樹，而且是同時算完的。

前面加 `~` = 把算完的結果反相

```
~&a  =  ~(&a)  =  ~0  =  1
~|a  =  ~(|a)  =  ~1  =  0
~^a  =  ~(^a)  =  ~0  =  1
```

[!WARNING] `a & b` 和 `&a` 是完全不同的兩件事，雖然都用 `&`：

寫法	幾個運算元	叫什麼	輸出寬度
<code>a & b</code>	兩個（符號在中間）	位元 AND	跟輸入一樣寬（4 位元）
<code>&a</code>	一個（符號在前面）	縮減 AND	永遠 1 位元

看到符號前面沒有東西，它就是縮減。

最常用的兩個：

```
wire is_zero = ~|a;      // a 全部都是 0 → 1    (比 a==0 省面積)
wire parity  = ^data;    // 同位元檢查，一行搞定
```

完整的六個縮減運算子和實際跑出來的值，在 [第 2 週的 06_operators_all](#)。

★ >>> 算術右移 —— Python 的 >> 其實就是這個

Python 的整數沒有固定寬度，`-4 >> 1` 直接就是 `-2`，你不用管補什麼。Verilog 的線有固定寬度，所以要你自己選補什麼。

```
a = 4'b1100;          // 當成無號 = 12，當成有號（二補數）= -4
```

寫法	左邊補什麼	結果	讀成有號	
<code>a >> 1</code>	補 0	0110	+6	X -4 除以 2 不會變 +6
<code>a >>> 1</code>	補符號位 1	1110	-2	✓ 對了

`>>>` 就是為了讓負數除以 2 在硬體上還是對的。Python 的 `>>` 對負數本來就是這個行為，只是 Python 幫你做掉了，Verilog 要你明講。

[!WARNING] `>>>` 一定要搭配 `signed`，不然沒有效果：

```
wire      [3:0] u = 4'b1100;
wire signed [3:0] s = 4'b1100;    // ← 要有 signed

u >>> 1 = 0110    ← 白寫，跟 >> 一樣
s >>> 1 = 1110    ← 這才對
```

編譯不會報錯，波形也不會變紅，它就只是安靜地算錯。

左移 `<<` 和 `<<<` 結果永遠一樣（左移不管有號無號都補 0），不用煩惱。

14. 動手跑一次

上面講的東西，這個範例全部跑一遍給你看：

```
cd week00_warmup
iverilog -o sim.out 02_syntax.v 02_syntax_tb.v
echo %ERRORLEVEL%
vvp sim.out
```

它會印出：常數三種寫法的對照、`{}` 四種用法的實際結果、`for` 迴圈算出來的東西。

建議做法：一邊看終端機輸出，一邊打開 `02_syntax.v` 對照原始碼。每一段輸出都對應檔案裡的一段程式。

改改看

改完存檔，把上面三行重跑（按 `↑` 上方向鍵叫回指令）：

1. 把 `02_syntax_tb.v` 第 28 行的 `a` 改成別的值，看 `{}` 的結果怎麼變
 2. 把 `02_syntax.v` 裡的 `{2{a}}` 改成 `{3{a}}` —— 但輸出只有 16 位元，猜猜會怎樣？
 3. 把某一行的分號刪掉，看錯誤訊息指到第幾行（故意製造錯誤是很好的練習）
-

最後：一頁小抄

```
// ----- 檔案骨架 -----
`timescale 1ns / 1ps           // 時間單位

module 名字 (
    input wire      a,          // 1 位元輸入
    input wire [7:0] b,          // 8 位元輸入
    output wire [7:0] y,         // assign 給值 → wire
    output reg  [7:0] z          // always 給值 → reg
);                                // ← 最後一個腳位不能有逗號

// ----- 組合邏輯 -----
assign y = a ? b : 8'd0;        // 三元運算

// ----- 縮減：一整條線縮成 1 位元 -----
wire all_ones = &b;              // b 全是 1 嗎      (Python: all)
wire not_zero = |b;              // b 有任何 1 嗎    (Python: any)
wire is_zero  = ~|b;             // b 全是 0 嗎
wire parity   = ^b;              // 1 的個數是奇數嗎

always @(*) begin
    z = 8'd0;                    // 先給預設值，避免 latch
    if (a) z = b;
end

// ----- 循序邏輯 -----
reg [7:0] q;
always @(posedge clk) begin
    if (rst) q <= 8'd0;          // 時脈區塊用 <=
    else    q <= b;
end

endmodule                        // ← 不用分號
```

看完這份就可以開始第 1 週了 → [../week01_basics/README.md](https://github.com/0x00sec/week01_basics/blob/master/README.md)